

Design a Rate Limiter You Can Defend

A rate limiter enforces a policy over an identity, resource, and time horizon. Fixed windows, sliding windows, and token buckets implement different semantics. Choose the behavior first, then reason about concurrency, regional limits, hot keys, clock behavior, and dependency failures.

For: Candidates who know common limiter names but need to specify the actual policy, atomic update boundary, distributed behavior, and user-visible failure semantics.

Suggested pace: 1-2 weeks of design and local simulation | Backend and concurrency basics

What you will be able to do

- Define the subject, scope, burst allowance, and charging unit of a limit.
- Compare fixed windows, rolling windows, and token buckets accurately.
- Make a local token update atomic and bound state growth.
- Explain distributed accuracy, availability, and failure-policy trade-offs.

Build the right mental model

Identity and cost come first

Limit by the relevant authenticated tenant, user, API key, endpoint, or another policy key. IP-only limits may group unrelated users or be unsuitable for the product. Some requests cost one unit; others need weighted cost based on bytes, tokens, or computation.

Algorithms express different promises

A fixed window can allow a boundary burst; a strict rolling count is more precise but may store more history. A token bucket allows bursts up to capacity while replenishing at a rate. It does not mean every sliding interval has a strict count cap.

Distributed enforcement is a coordination choice

Several gateways updating one key need an atomic shared operation or a deliberate approximate/local-budget design. Global precision costs coordination and latency. During a partition, a fail-open policy and fail-closed policy have different business consequences.

Study with evidence.

Write a policy in plain language and derive the algorithm from it. Simulate boundary times and concurrent arrivals with a controllable clock. Treat illustrative rates as exercise inputs, not recommended universal production settings.

Sessions 1-3: Policy and algorithms

Define limit keys, request costs, burst allowance, response behavior, and scope. Compare fixed window, sliding log/counter, and token bucket using short timelines near boundaries.

Deliverable and check: A policy table showing which algorithm meets each promise and where approximation is accepted.

Sessions 4-6: Atomic local behavior

Implement refill-and-consume as one protected operation. Use a monotonic elapsed-time source where appropriate, clamp tokens to capacity, and handle requests whose cost exceeds available tokens.

Deliverable and check: Deterministic tests for simultaneous requests, long idle periods, and insufficient capacity.

Sessions 7-9: Multiple gateways

Choose shared state, regional budgets, or local approximation. Analyze hot keys, expiration, missing identity, and the failure of the state store. Define a bounded fallback rather than silently disabling protection.

Deliverable and check: A distributed design with explicit accuracy and availability consequences.

Sessions 10-12: Observe and tune

Measure accepted/rejected counts, limiter latency, state-store load, and tenant fairness. Add safe configuration rollout and test key cardinality. Document retry hints and avoid encouraging synchronized client retries.

Deliverable and check: A small load simulation and an operational policy for overload, rollback, and support investigation.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Trace a token bucket and expose a race

A bucket has capacity 10 tokens and refills at 2 tokens/second. Each request costs one token. It starts full, and 10 simultaneous requests arrive at time zero, followed by more requests half a second later.

A worked approach

1. At time zero, at most 10 one-token requests should succeed. The update must atomically read tokens, compute refill, decide, and consume; separate unsynchronized reads can let several requests spend the same token.
2. After 0.5 seconds, one token has replenished, so one additional request can succeed under this idealized timeline. After a long idle period, the stored amount is capped at 10 rather than growing without bound.
3. Repeat the test through two simulated gateways. A single shared atomic bucket preserves the stated count; independent full buckets would allow twice the burst unless the policy intentionally partitions capacity between them.

Expected result

The trace demonstrates burst and refill semantics, while the concurrency test exposes the true enforcement boundary. The design's distributed guarantee follows from where the atomic state lives.

Verify the result

- Can two requests consume the same remaining token?
- Is refill capped and based on a safe elapsed-time calculation?
- Does a state-store failure follow the documented policy?

Extend the exercise

Make one request cost six tokens and another cost one. Explain fairness, rejection, and maximum request cost, then consider a separate concurrency limit for long-running requests that a rate limit alone cannot bound.

Make the learning stick.

Is a rate limit the same as a concurrency limit?

No. Rate controls arrivals over time; concurrency controls simultaneous in-flight work. Slow requests can exhaust resources even under a modest rate, so some services need both with coordinated admission rules.

Should a limiter fail open?

It depends on the consequence. Some availability-focused reads may tolerate a bounded fallback, while expensive or security-sensitive operations may need stricter rejection. Make the decision explicit and observable rather than universal.

Why can a fixed window allow a double burst?

Requests near the end of one window and the start of the next are charged to different counters. A short real-time interval spanning the boundary can therefore contain nearly twice the nominal per-window count.

Avoid these traps

- Selecting an algorithm before specifying the policy.
- Using non-atomic read-modify-write for shared counters.
- Assuming per-region limits automatically enforce a precise global limit.

Ready for the next step?

- Identity, cost, time horizon, and burst semantics are explicit.
- Concurrent updates preserve the policy.
- Regional and failure behavior are documented.
- Tests cover boundaries, weighted requests, and clock assumptions.

Reliable next reads

- [Azure rate limiting pattern](#)
- [Redis rate limiter patterns](#)
- [RFC 6585: additional HTTP status codes](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.